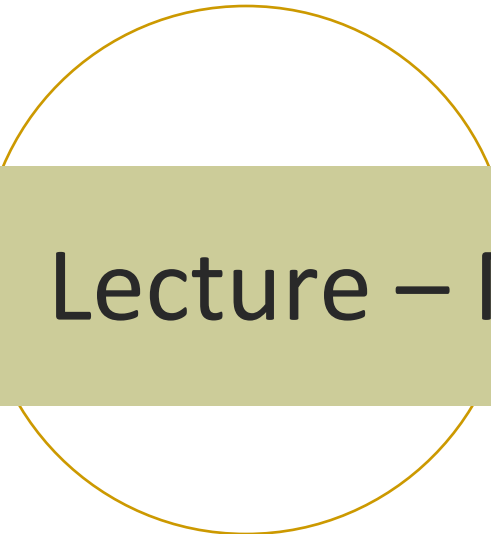




Lecture – Memory characteristics

Nothing is more responsible for the good old days
than a bad memory.

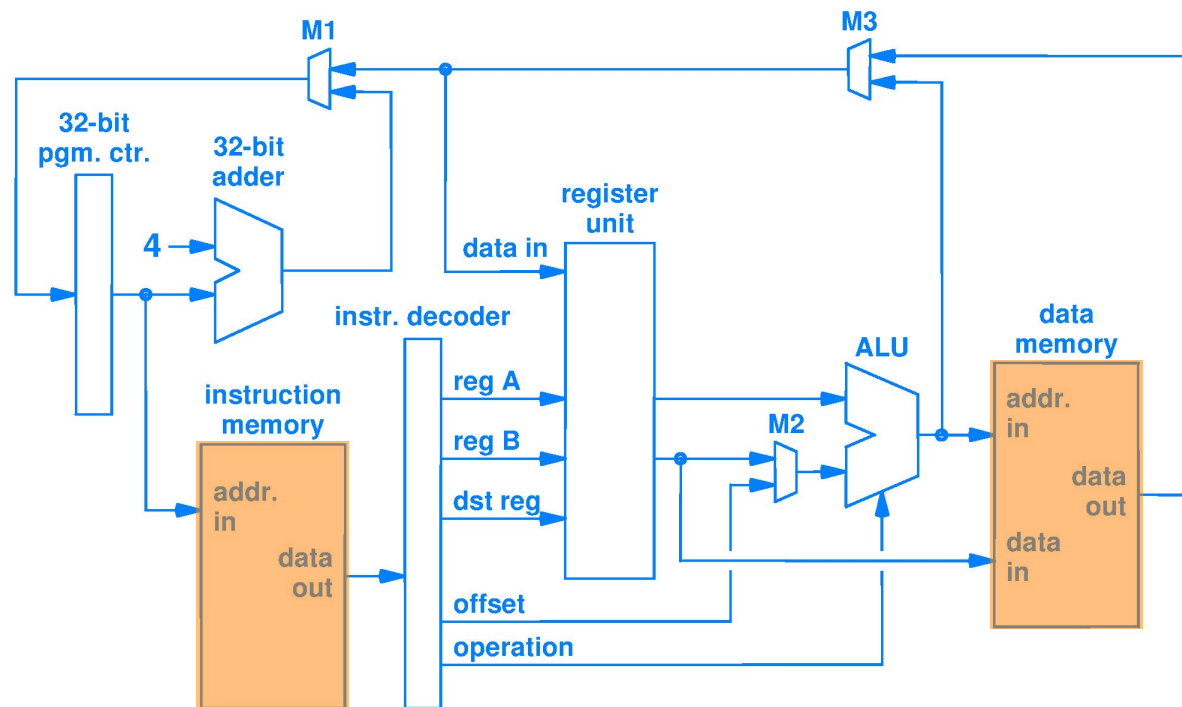
– Franklin Pierce Adams

What is the big picture for memory?

- Read Chapter 5
- Learning objectives
 - Define 3 kinds of memory, Fetch-Store paradigm, RAM, sequential access memory

Memory technologies

- Design goals
 - Store program and data
 - Support program execution well



There are different kinds of memories

- Registers – the few, the fast; tiny pointer size
- Main memory – holds running programs and their data; full-sized pointer
- Storage – non-volatile repository; operating system constructs the “pointer” which may not be an unsigned integer, e.g., hard disk block “coordinates”
- Computer architects organize the above into a hierarchy of costs and capacities

Fetch-Store paradigm

- All memory technologies may be modeled as performing
 - Load (fetch): $\text{value} \leftarrow \text{memory}[\text{address}]$
 - Store (write): $\text{memory}[\text{address}] \leftarrow \text{value}$

CPU and memory technology overview

- Logic built as silicon chips, predictable progress
 - *International Technology Roadmap for Semiconductors*
- Memory technologies since 1960s: punched paper, trapped sound, magnets, logic gates, ¼-wave reflectors, capacitors, others
 - Memory progress is less predictable
 - Memory systems are complex
 - Good news: Details mostly hidden from software
 - **Bad news:** Performance is not hidden from software

Memory access latency

■ Latency

- Elapsed time from start to completion of a memory operation (read or write)
- May depend on the preceding memory operation
- Primary consideration for performance of instruction fetch and data read and write

Read versus write latency

- For many memory technologies
 - Time to write (store) > time to read (fetch)
 - Difference can be large
- Consequence: meaningful measure of performance must give both latency times
- Device datasheet must discuss whether one operation affects its following operation

Read only memory (ROM)

- Most memory technologies support both fetch (read) and store (write) symmetrically
- Some technologies support unlimited fetch but write one time or a “few” times
- Such an asymmetric memory is called ROM, for “read only memory”
 - Used for unchanging or rarely changing bit strings
 - Used to prevent end user from making unauthorized changes to software/data

Access paradigm, or Is latency = $f(\text{location})$?

- Random access memory (RAM) – latency is *approximately constant*, i.e., latency $\neq f(\text{location})$
 - Best case and worst case times are about the same
 - Choice of algorithm or data structure can be made without concern for memory performance
- Sequential access memory – latency is a *function of the “distance” to the location*
 - “Near” locations can be accessed more quickly than “far” locations: **seek time**
 - Example: FIFO queue (first-in-first-out)
 - Algorithm and data structure choices must consider location or performance may be exceptionally poor

4 key memory characteristics

- Read Chapter 5
- Learning objectives
 - Define volatility, density, cost, and capacity for various memory technologies
 - Explain the purpose of a memory controller

Volatility

■ Volatile

- Bit strings are forgotten when power is removed; Remembering requires power continuity
- Often, volatile technologies have low latency

■ Non-volatile

- Bit strings inaccessible when power is removed, but not forgotten
- Remembering does not require power
- Restoring power restores access

Density

- Usually expressed as the total capacity of a single device, e.g., 1 Gbit chip, 1 TByte disk
- For volume-sensitive devices, density will refer to bits per unit area or per unit volume, depending on which makes more sense for a given technology
 - High density is desirable

Cost

- Cost is a very important criterion for memory technologies
- There is often good reason to wish for larger capacity, but cost may be the major limiting factor, rather than density, heat, noise, ...

Capacity

- Measured in units of bytes
- Some technologies are 2^k -byte capacity, inherently, some are not
- Non-power-of-2 capacity examples
 - Punched card (capacity $\approx$ convenient card size)
 - Magnetic tape ($\approx$ convenient tape length)
 - Hard disk ($\approx$ feasible platter diameter and number)
 - CD, DVD, and Blu-ray (all disks same physical size, area/bit sized for both reasonable cost and so disk can store largest intended content)
- Non- 2^k capacity is reported using powers of 10

Processors speak Pointer. Memory? Not so much.

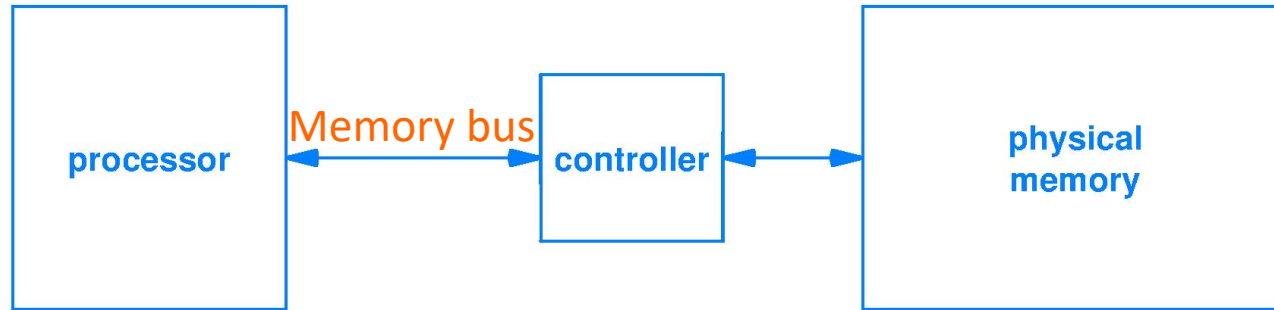


Figure 11.3 Illustration of the hardware used for memory access. A controller sits between the processor and physical memory.

■ Memory controller

- An interface to manage various incompatibilities between the processor and a physical memory device
- So that software may use pointers, controller translates them to addressing used by the memory technology
- Controller also manages the latency mismatch between processor circuit and the memory technology

Historic memory technology examples

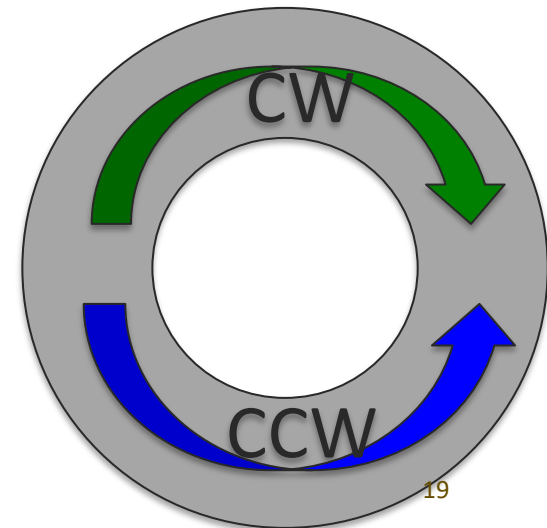
- Learning objectives
 - Gain familiarity with punched card, acoustic tube, Williams tube, and magnetic core memory

Punching punched cards



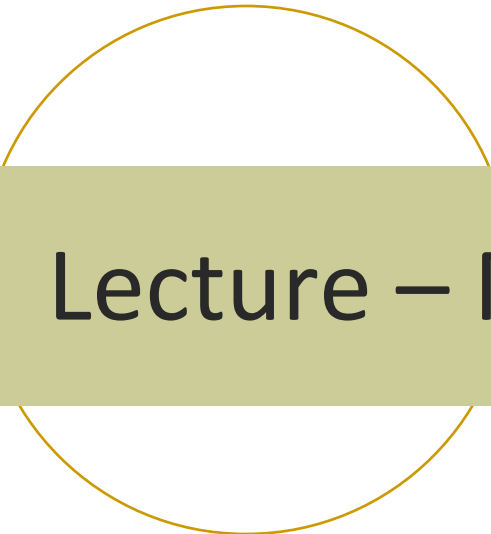
Core: non-volatile, RAM, historic

- Origin of the term “core dump”
- Bit stored as a clockwise (CW) or counter-clockwise (CCW) magnetization of a ferrite toroid; a true digital medium
- EMP and radiation “hard”
- *Interesting characteristic:* Reading a bit is done by writing a 0 to that bit (reset) and sensing how much energy was needed (called a **destructive read**); Must re-write 1 bits after reading
- Manufactured circa 1955 - 1975





Close up of core memory of the CDC 6600 computer used at Purdue for many years. Cores were threaded by hand onto the addressing wires (horizontal and vertical) and the sensing wire (diagonal).



Lecture – Memory technology today

With today's technology you can make memory fast, large, and affordable, so long as you pick just two.

– *Unknown*

Volatile semiconductor memory

- Learning objectives
 - Understand the SRAM and DRAM memory cell
 - Understand SRAM and DRAM operation

Semiconductor memory – volatile, RAM

- Built using CMOS or MOS processes
- Bit is stored in a **memory cell circuit**
 - *Different memory cell types correspond to different families of semiconductor memory*
- Much faster, denser, cheaper than core
 - Volatility not a significant drawback, typically
 - Quickly replaced core for most applications
 - Core still used by computers in ionizing radiation and/or EMP environments

SRAM – fast, volatile, 2^k , CMOS

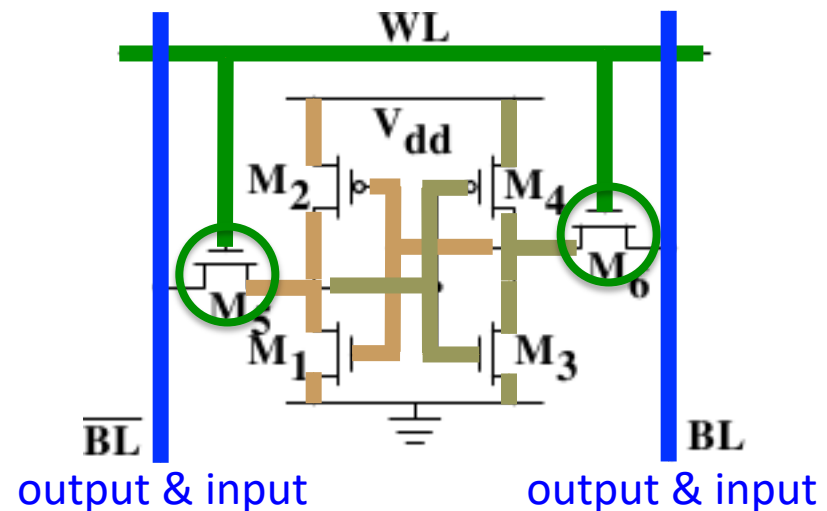
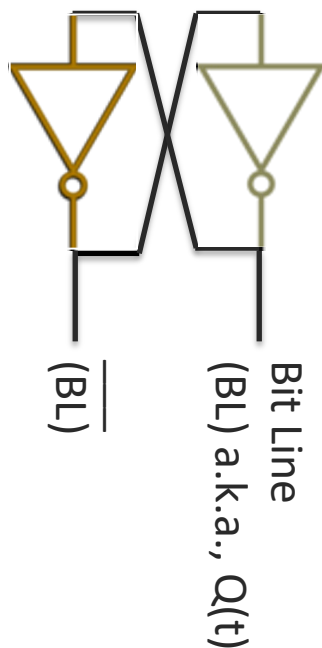
- SRAM (Ess-ram, static RAM)) since late 1960s
- Bit strings remembered if power is “static” (On)
- Latency somewhat greater than register latency
- Today, SRAM is
 - physically on same CMOS chip as processor circuit
 - the technology implementing **cache memory**

Two-dimensional addressing

- One k -to- 2^k decoder points to 2^k locations; if, say $k=32$, then decoder circuit is very large
- Integrated circuits are 2-dimensional (2D)
 - Arrange 2^k locations in a $\sqrt{2^k}$ by $\sqrt{2^k}$ 2D array
 - Use pair of $(k/2)$ -to- $2^{k/2}$ decoders to point to $2^{k/2}$ **row-select wires** and $2^{k/2}$ **column-select wires** to define 2^k row/col intersection locations
 - 2D layout reduces decoder circuit chip area (space complexity) from $O(2^k)$ to $O(2^{(k+2)/2})$
 - Example, $2^{32}/2^{(32+2)/2} = 2^{15}=32,768$ times less ckt area
 - Chip cost $\propto$ chip area

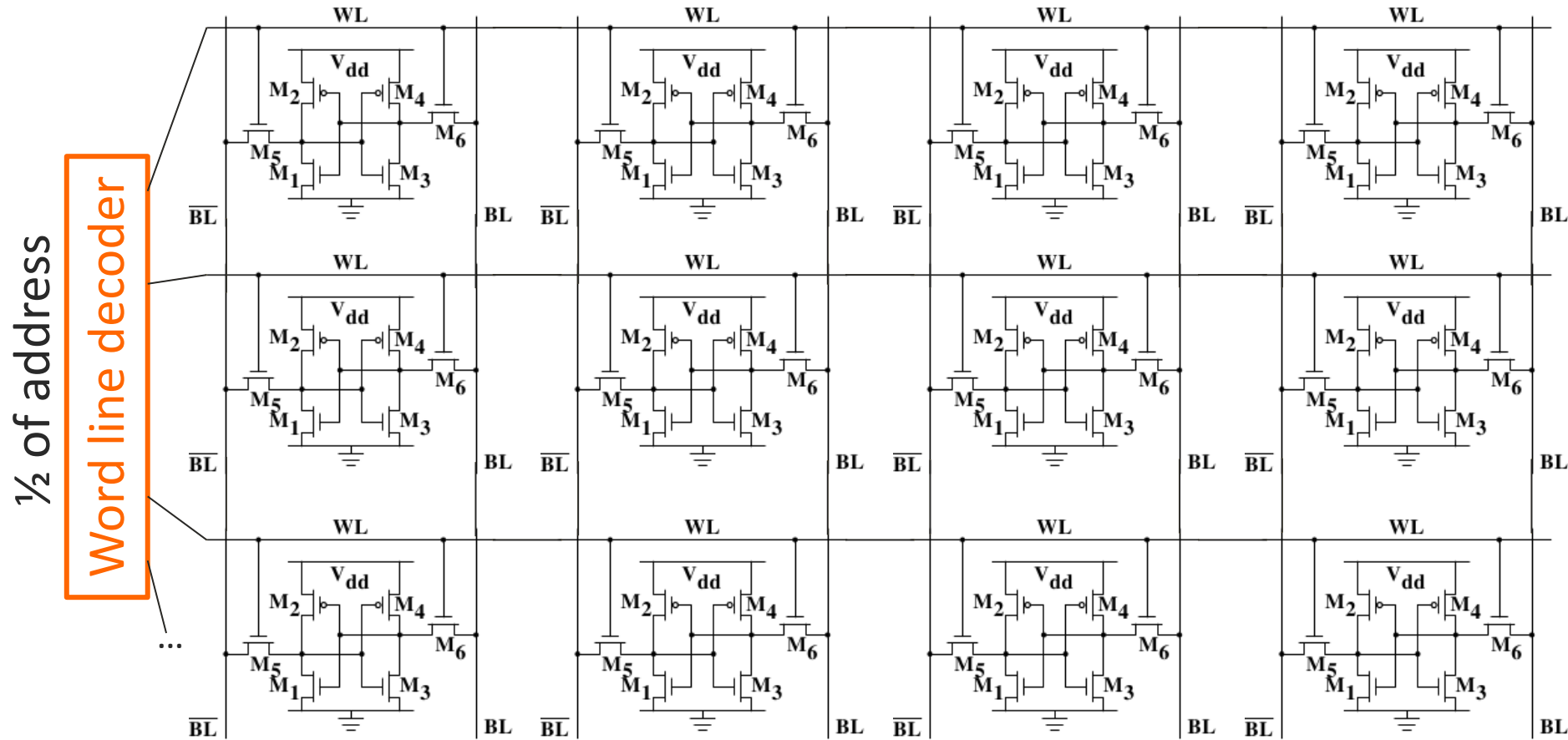
SRAM memory cell circuit – store 1 bit

- Inverter latch circuit with 2 transistors to link inverter outputs to input/output buses
- 6 transistors/bit (cf. 26/bit for edge-triggered D flip-flop)



SRAM bit-cell array on a chip

- Word line mux selects all of one $2^{k/2}$ -bit array row, column mux/demux selects 1-bit string from within array row bit string



$\frac{1}{2}$ of address

Column mux/demux for read/write via bit lines (BL)

Advantages, disadvantages of SRAM

■ Pro

- Access time independent of location (address); RAM
- Address with unsigned integer (simple, fast controller)
- Low latency of circuit & low latency of bit string transport because on same chip as processor
- Low power of communication because on same chip

■ Con

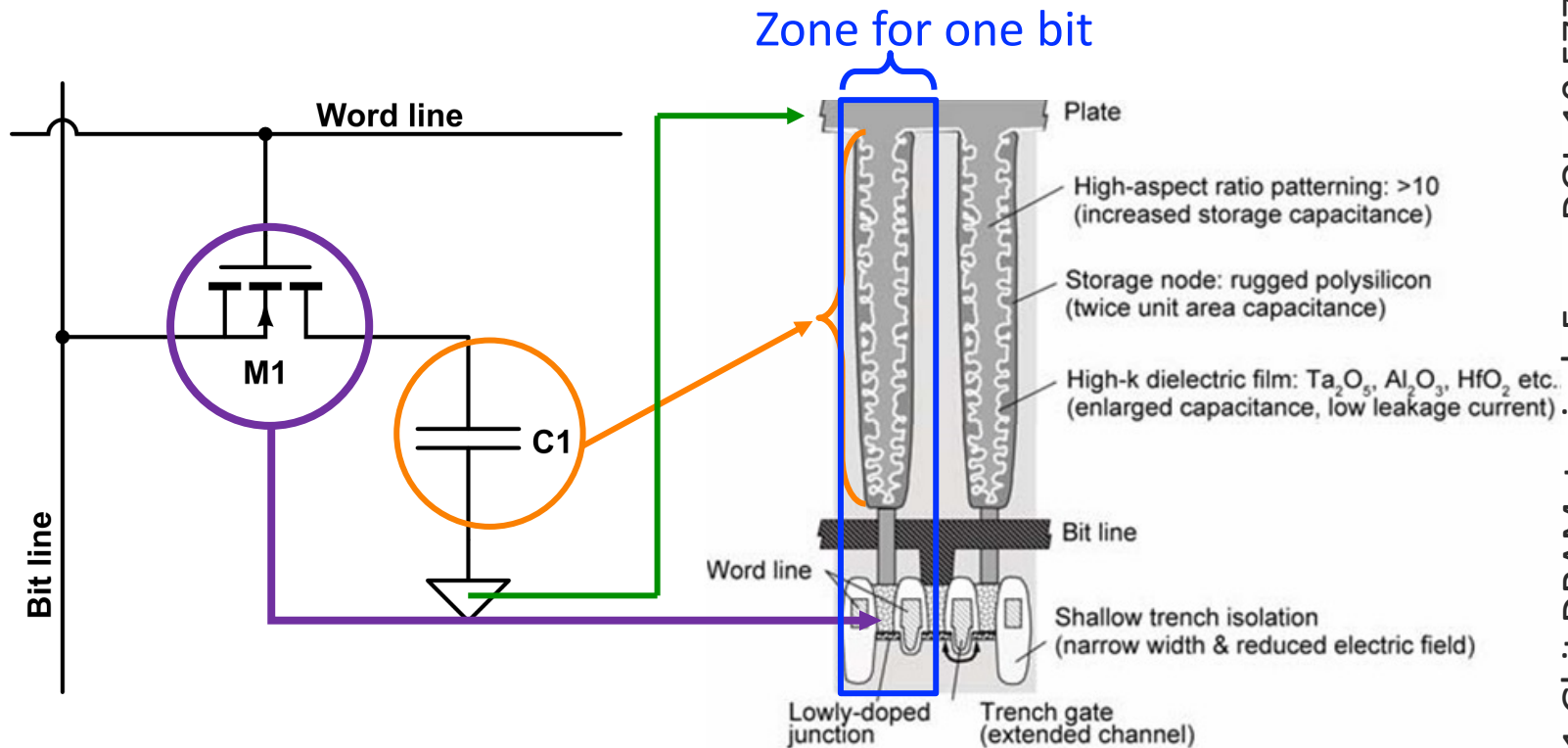
- High-ish memory function power – per bit power consumption falls between registers and DRAM
- Inside a high-cost package for the hot processor circuit
- Modest density – more than 4x better than registers, but too low for today's massive memory capacity demands
- Processor manufacturer sets SRAM capacity

DRAM – volatile, 2^k , ckt is MOS + capacitor

- DRAM (DEE-ram) – Dynamic RAM
- Logic 1 represented by charge stored in a capacitor, 0 by discharged capacitor
- 1972: Intel 1103 DRAM chip at \$0.01/bit; displaced core memory in a few years
- Unfortunately, DRAM capacitors self-discharge to zero charge in ~ 100 msec at 80°C
 - Thus, *power supply energy alone is not enough to remember a bit*
 - Must also **refresh** each stored charge often

DRAM memory cell circuit – store 1 bit

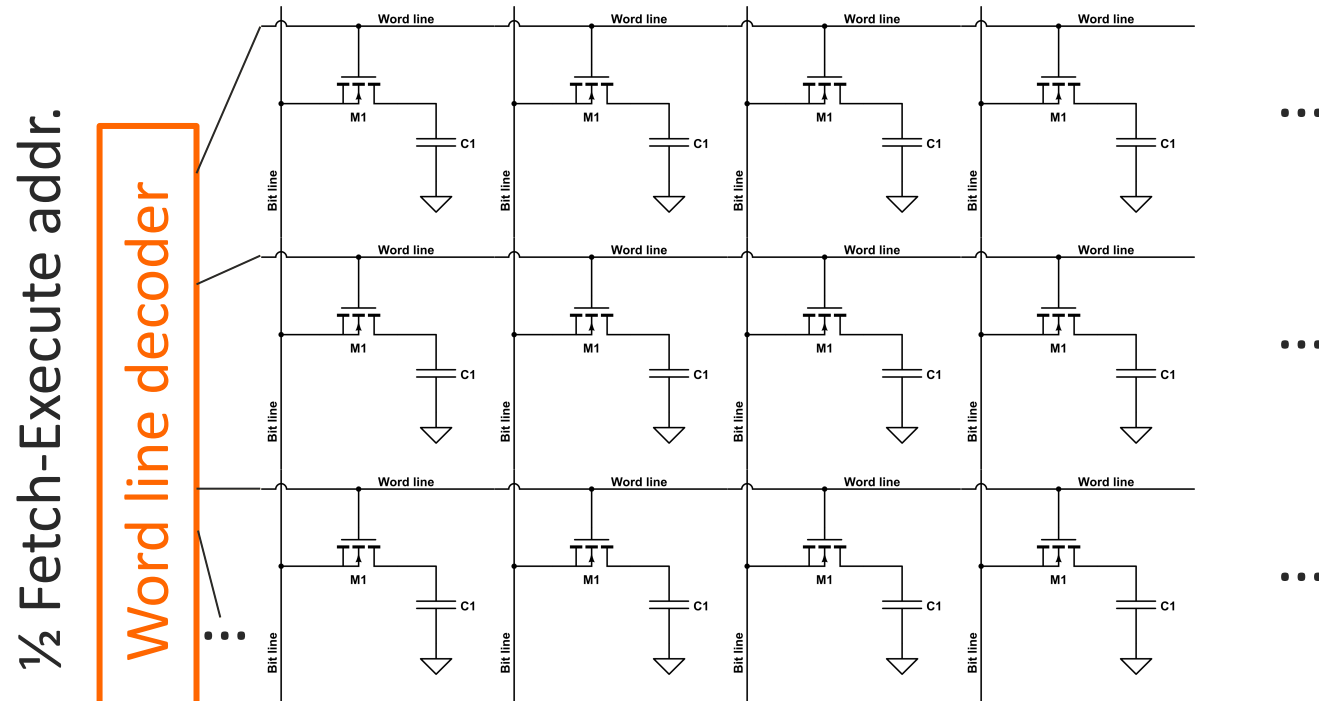
- DRAM cell: one MOSFET and one capacitor
 - Passive storage device (capacitor) $\Rightarrow$ Low power
 - Vertical cell $\Rightarrow$ small chip area $\Rightarrow$ Dense storage



1 Gbit DRAM, typical. From DOI: 10.5772/8638
By Hideo Sunami. Published: April 1st 2010.

DRAM bit cell array on chip

- Word line mux selects all of one $2^{k/2}$ -bit array row, has F-E address input and Refresh circuit input; Column mux/demux selects 1-bit string from within array row bit string



$\frac{1}{2}$ F-E addr.

Column mux/demux for read/write via bit lines (BL)

Advantages, disadvantages of DRAM

■ Pro

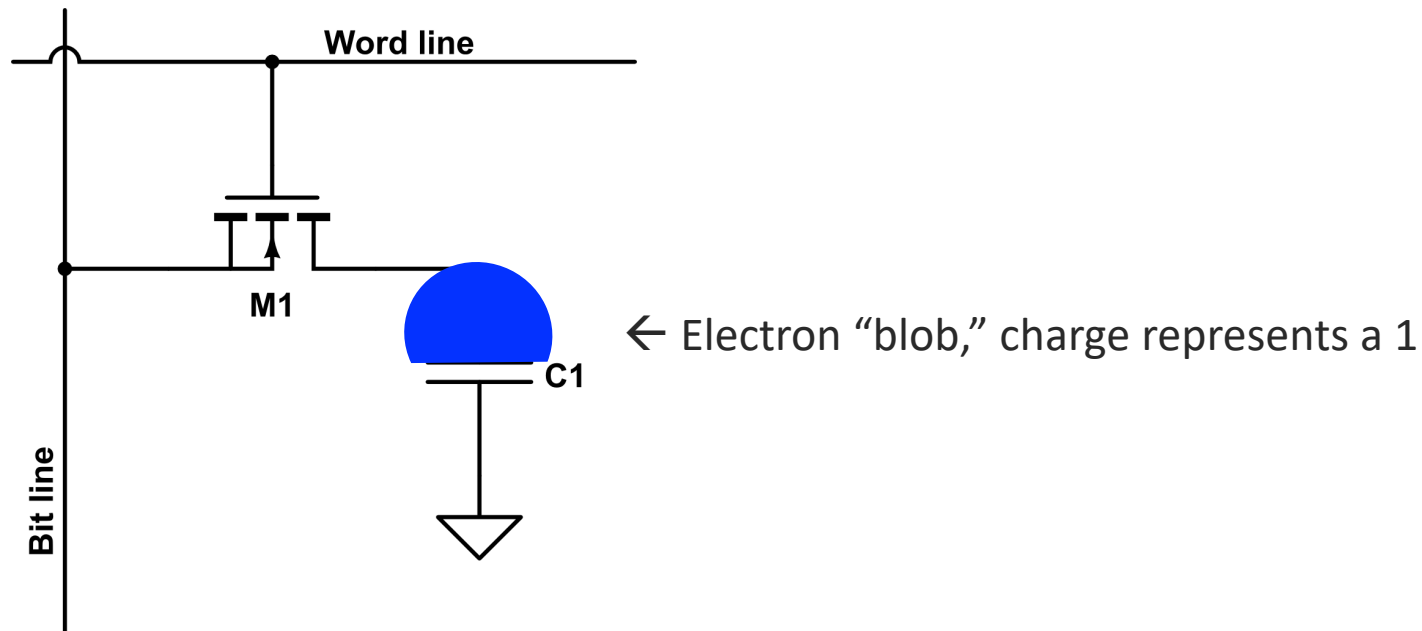
- Access time independent of location (address); RAM
- High density
- Low power
- Address with unsigned integer (no translation needed)
- Low cost because of high density and low power

■ Con

- High latency – sensing charge versus no charge takes much more time than sensing a voltage level
- Requires controller circuit to refresh (re-write charges to reference value) because DRAM memory cell circuit is inherently analog, not digital

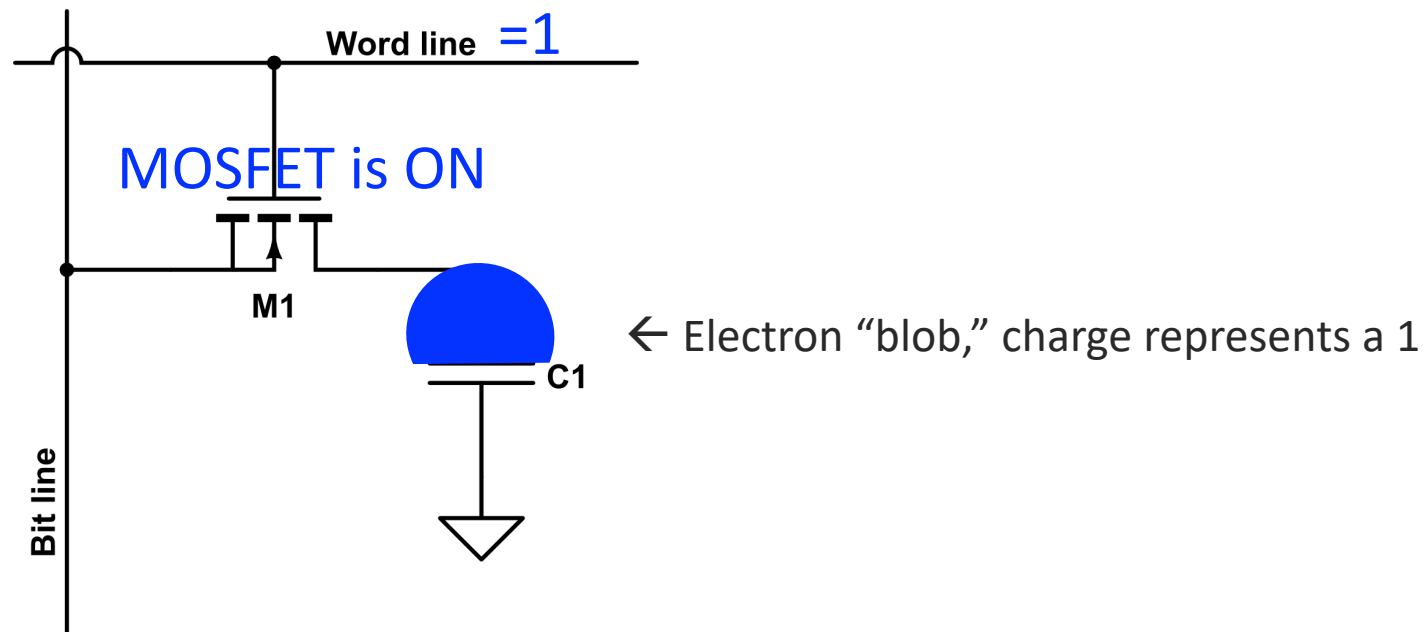
Reading DRAM memory cell (1 of 4)

- Select word line, *use Bit Line as output bus*
 - M1 turns On, allows charge transfer from C1
 - Sense presence or absence of charge on Bit line
 - After read, re-write if 1-valued bit was read



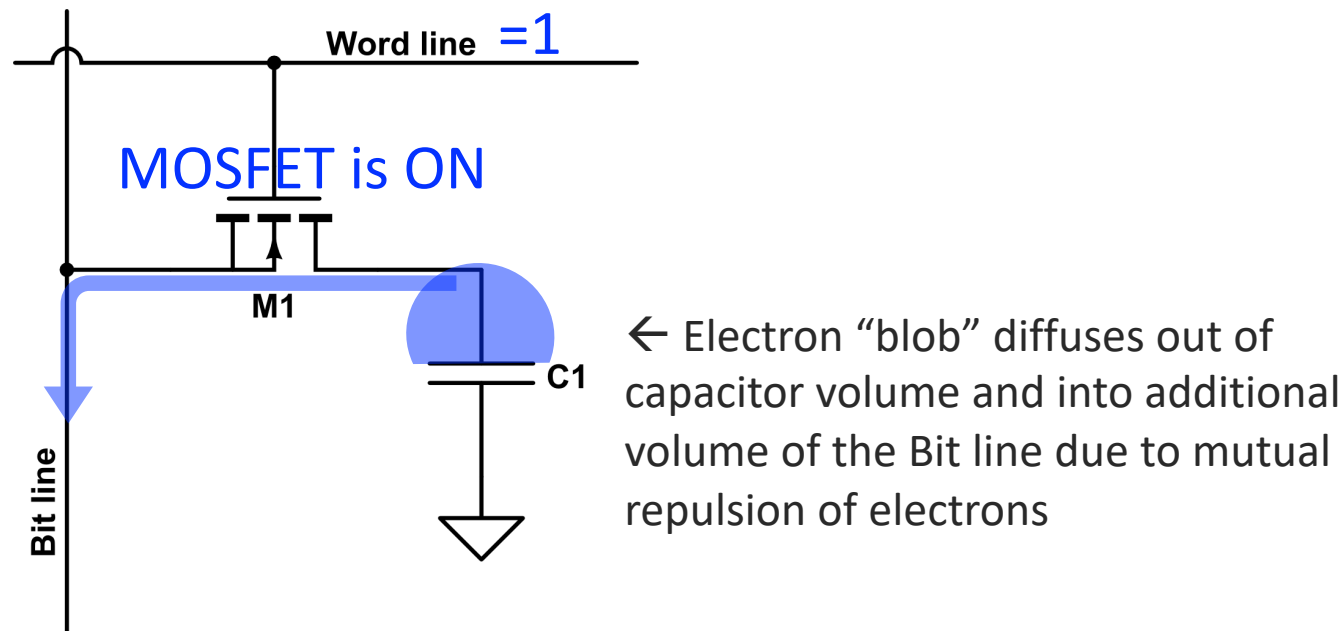
Reading DRAM memory cell (2 of 4)

- Select word line, *use Bit Line as output bus*
 - M1 turns On, allows charge transfer from C1
 - Sense presence or absence of charge on Bit line
 - After read, re-write if 1-valued bit was read



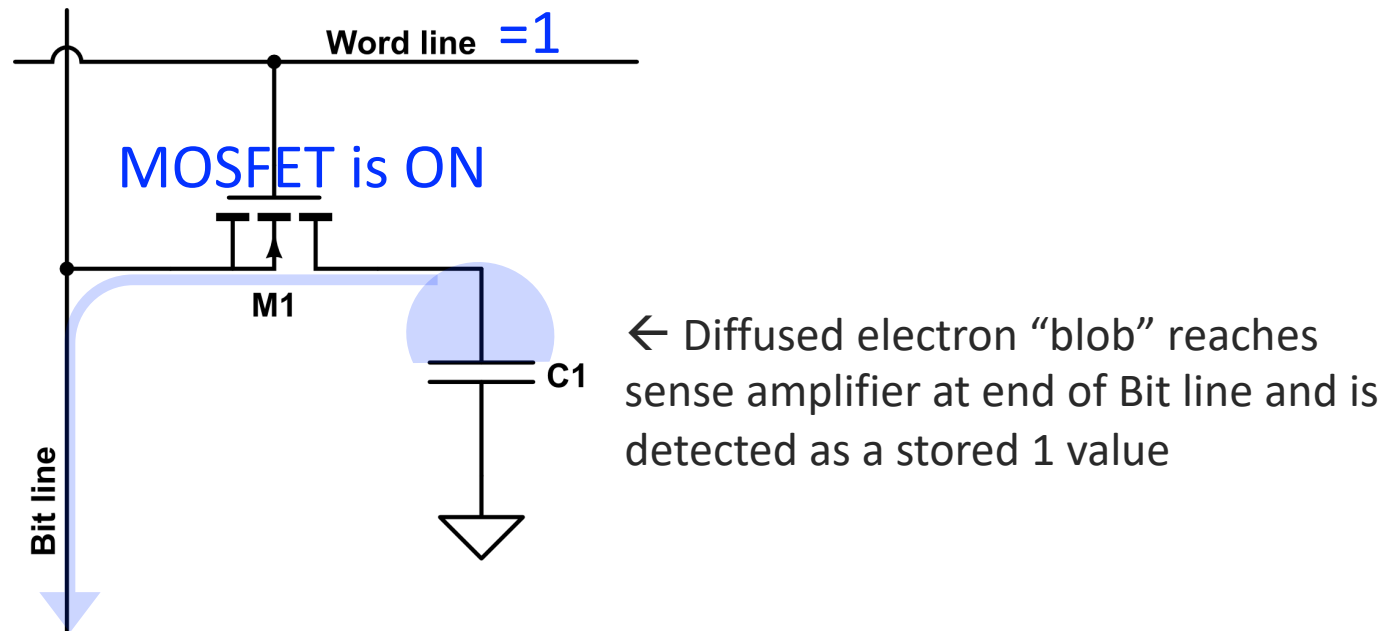
Reading DRAM memory cell (3 of 4)

- Select word line, *use Bit Line as output bus*
 - M1 turns On, allows charge transfer from C1
 - Sense presence or absence of charge on Bit line
 - After read, re-write if 1-valued bit was read



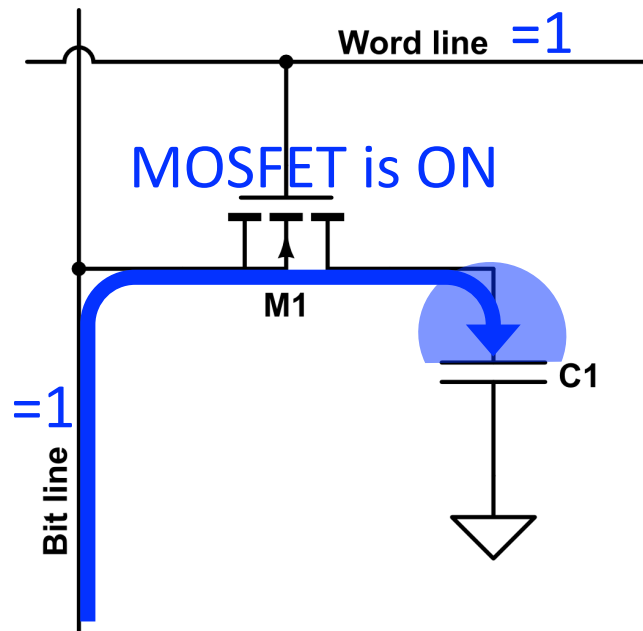
Reading DRAM memory cell (4 of 4)

- Select word line, *use Bit Line as output bus*
 - M1 turns On, allows charge transfer from C1
 - Sense presence or absence of charge on Bit line
 - After read, re-write if 1-valued bit was read



Restore 1 after destructive read (1 of 2)

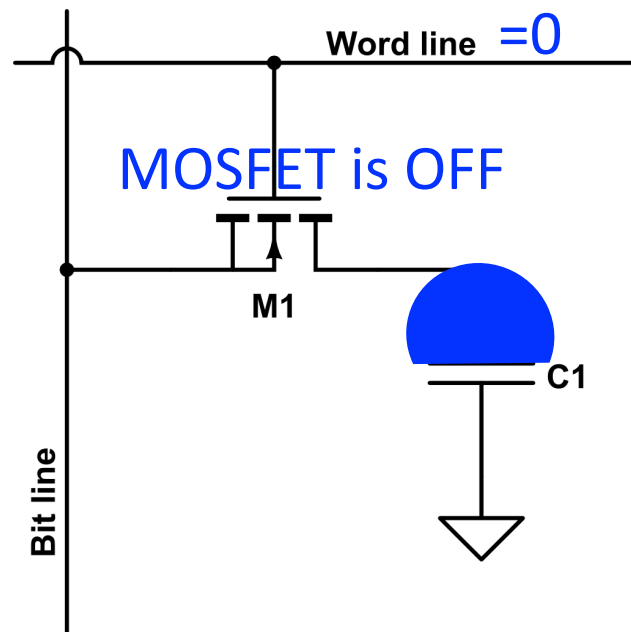
- Select word line, *use Bit line as input bus*
 - M1 turns On, allows charge transfer to C1
 - Bit line supplies the restore charge



Charge in capacitor is increasing to restore a 1

Restore 1 after destructive read (2 of 2)

- Select word line, *use Bit line as input bus*
 - M1 turns Off, blocks fast charge transfer to/from C1; now charge only leaks out of C1
 - Bit line disconnected from power supply



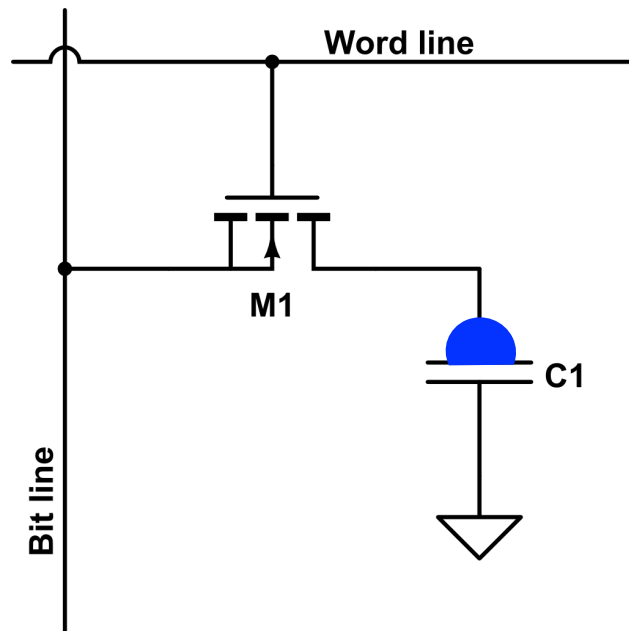
Charged capacitor again represents 1 value; begins to leak charge, again 😬

DRAM controller refresh algorithm

- `while(1) {`
 - read bit string at Word line with address J into a register circuit
 - write back the just-read Word line bit string, recharging all the discharged 1-valued bits
 - $J = (J+1) \text{ modulo } 2^{\text{number_of_word_line_address_bits}}$
 - `}`
- Characteristics
 - Refresh accesses in addition to fetch-execute accesses
 - Controller must not cause program correctness issues, so given higher priority than program memory access
 - Controller and DRAM use a little energy continually

DRAM refresh: sustaining 1 bits (1 of 4)

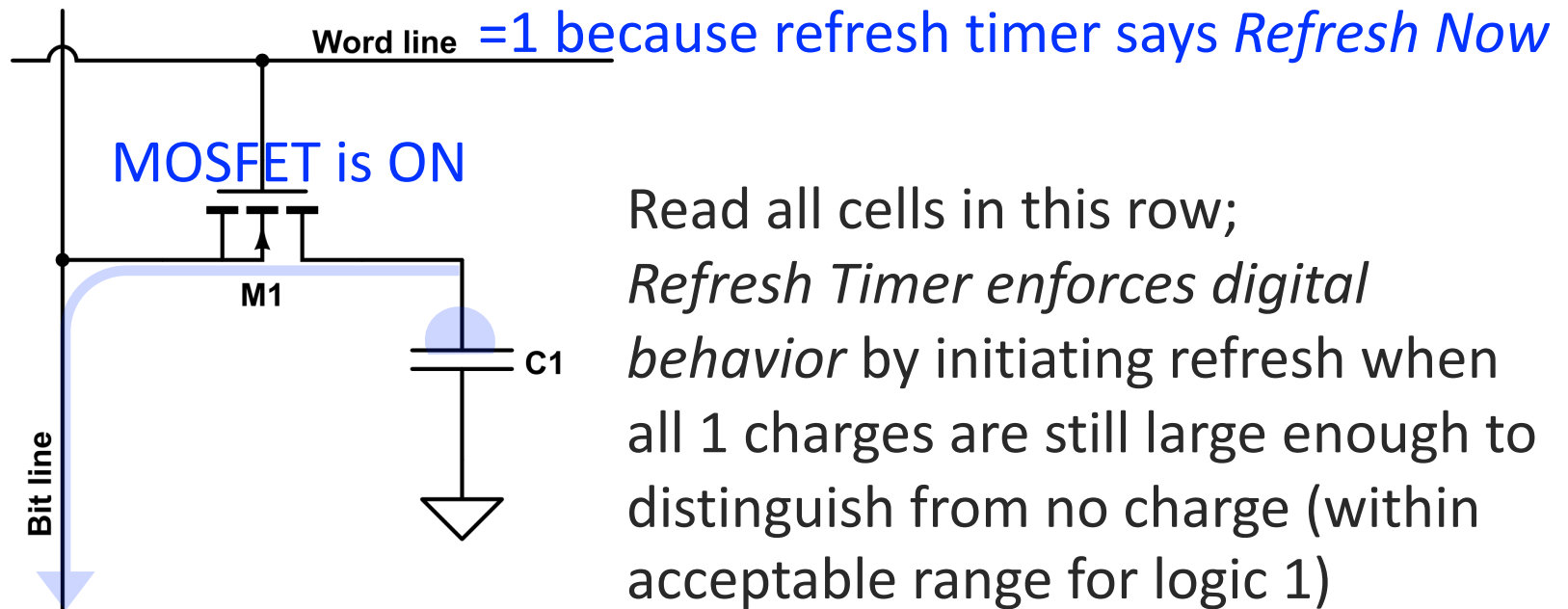
- Charge leakage overcome by **refresh**
 - Re-write all 1s every 64 msec (JEDEC standard)
 - Uses ~1% of time & noticeable amount of energy
 - Programs cannot access memory during refresh



Charge of logic 1,
partially leaked,
must refresh

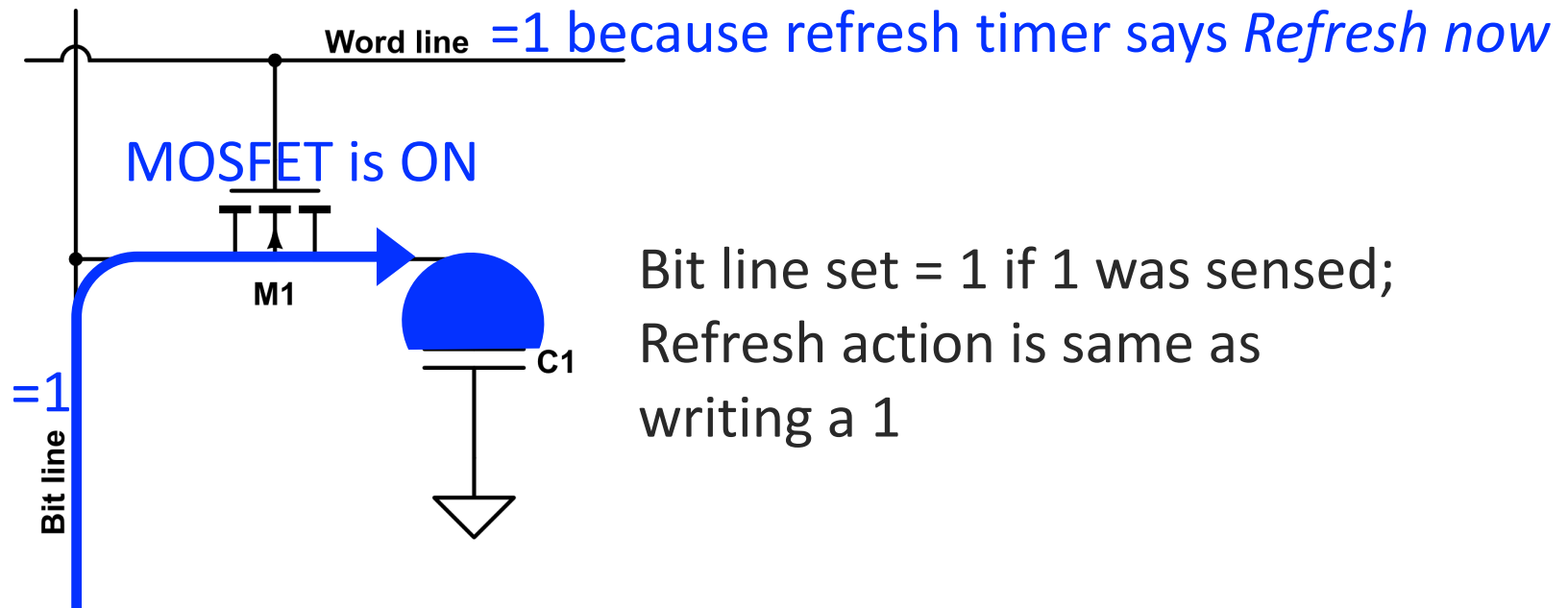
DRAM refresh: sustaining 1 bits (2 of 4)

- Charge leakage overcome by **refresh**
 - Re-write all 1s every 64 msec (JEDEC standard)
 - Uses ~1% of time & noticeable amount of energy
 - Programs cannot access memory during refresh



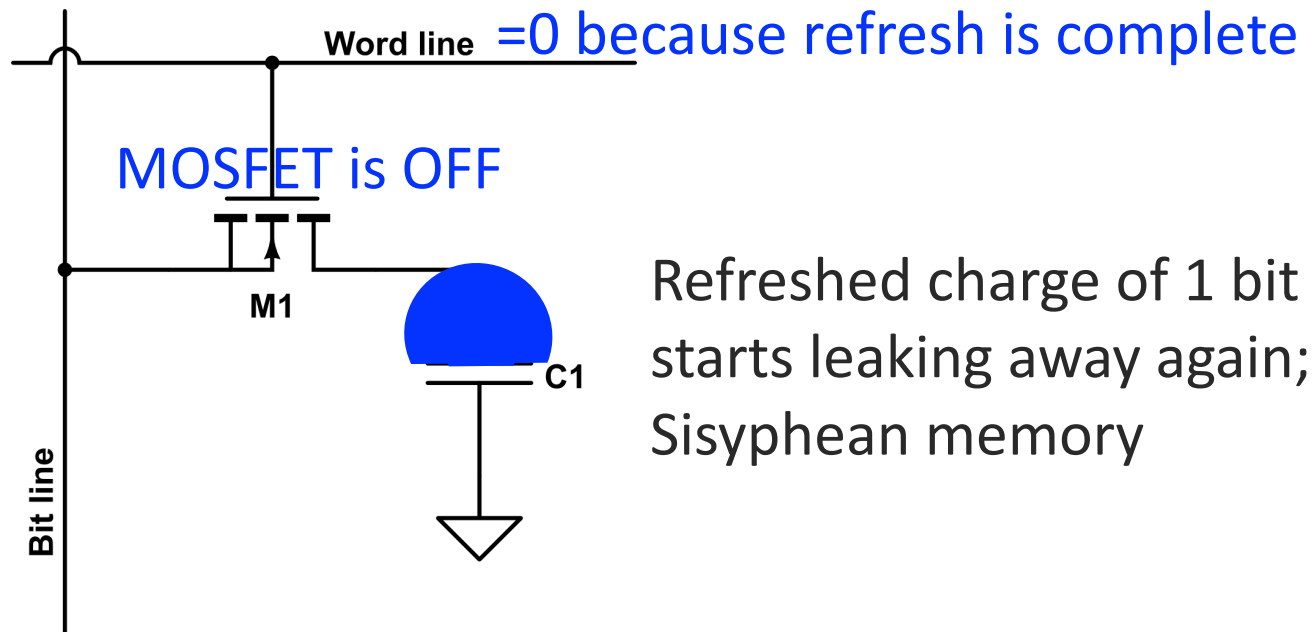
DRAM refresh: sustaining 1 bits (3 of 4)

- Charge leakage overcome by **refresh**
 - Re-write all 1s every 64 msec (JEDEC standard)
 - Uses ~1% of time & noticeable amount of energy
 - Programs cannot access memory during refresh



DRAM refresh: sustaining 1 bits (4 of 4)

- Charge leakage overcome by **refresh**
 - Re-write all 1s every 64 msec (JEDEC standard)
 - Uses ~1% of time & noticeable amount of energy
 - Programs cannot access memory during refresh



DRAM memory, a joy to behold,



but paddling like hell underneath
to make it happen.

DRAM redundancy (at-factory repair)

- When DRAM chip capacity $\sim \geq 64$ Kbit, spare rows and columns are included in array
 - More, denser cells correlates with greater probability of a non-functioning cell
 - Row/column spares allow “map out” of bad bits during chip testing at the factory
 - Decoder circuits enhanced to permit replacing malfunctioning row and column with spares
 - One way to alter decoder: apply excess power to blow fuses on chip, changing pointing logic circuit
 - Spares can repair a few bad cell(s) at the factory

Non-volatile memory today

- Learning objectives
 - Understand flash and hard disk technologies
 - Understand technology competition between flash and hard disk

Flash – non-volatile, RAM, 2^k , MOS

- Memory cell is a transistor “floating gate,” a very, very slowly leaking capacitor
- No refresh needed, thus no refresh power used
- Number of cell charge levels may be 2, 4, or 8 to store 1-bit/, 2-bits/, or 3-bits/cell
- High demand has supported intensive commercial efforts to improve this technology
- Flash memory is the technology implementing
 - Solid State Disk (SSD), SD cards, thumb drives

Flash characteristics and its self-repair

■ Pro

- Access time independent of location (address); RAM
- Very high density, high capacity, low-ish cost
- Low latency, power, noise compared to other non-volatile

■ Con (all minor, actually)

- Memory cells wear out during end user operations
 - Floating gate insulation (glass) eventually cracks when writing due to crystal strain; crack allows charge to leak from cell quickly
 - Circa 10^5 to 10^6 set/erase cycles before glass surely cracks
- Complex memory controller required
 - Implements wear-leveling to reduce wear out failures
 - Controller writes always into next higher free address modulo 2^k
 - Controller migrates existing data through entire address space
 - *Maps out cracked cell when fault detected; Capacity shrinks; Repairs happens in the field, automatically, i.e., in the hands of the end user*
 - Read, write, and erase operations are each different and complex
 - Example: Reading disturbs adjacent locations' charge amounts: must count accesses and "refresh" nearby data after some number of reads

Hard disk – non-volatile, sequential, non-2^k, electro-mechanical

- Introduced in 1956 by IBM
- Bit stored as directionally magnetized area on a thin film of media coating on rigid (hard) rotating surface
 - Direction of magnetization determines bit value, 0 or 1
 - Media permits unlimited read/write cycles; no need for wear leveling
- Density today > 1 Terabit/inch² achieved by
 - Ultra-precise mechanical part shapes and positional control, and
 - Exceptional analog signal processing, analog to digital conversion, plus error detection/correction encoding of data
 - Essential to overcome weak, noisy, analog signal read by head from the tiny magnetized regions

Hard disk – internal mechanism, 3.5"



Hard disk – controller circuit (green board)



Hard disk addressing and latency

- To address a given location on a platter
 - Read/write heads must be positioned over the intended concentric track (8ms avg. positioning latency) and
 - Platter must rotate until desired track sector (block) is under head, then the data can stream to read or write

Spindle RPM	Avg. Rotational Latency (msec)	Relative Improvement
5400	5.56	0%
7200	4.17	33%
10000	3.00	85%
15000	2.00	178%

- Flash ended the market for 15,000 & 10,000 RPM

Hard disk key trends

- Density
 - Improved 60%/yr 1956 – 1990, 100%/yr 1990 – 2005, today about 25%/yr: factor of 10^9 overall
- Platter diameter
 - too small a diameter means hard disk loses on cost per bit with respect to flash
 - recording density improvements end need for larger diameter platters (12-inch was once common)
- Resulting diameter squeeze leaves only two diameters in production in 2025
 - 3.5 inch for desktop to warehouse-scale computers
 - 2.5 inch for laptop computers (becoming rare)

Hard disk advantages & disadvantages

■ Pro

- Inexpensive
- Dense – circa 10 TBytes in 30 cm³ volume

■ Con

- High latency
- Moderate power required
- Destroyed by mechanical shock when operating
- Mechanism wears out in a few years (~3-5) of continuous use, causing access failure

Magnetic Tape: non-volatile, sequential, non- 2^k , electro-mechanical

- Magnetizable particles attached to long strip of plastic
 - Read/write performed by electromagnetic "head"
 - Moving tape is in sliding contact with head
 - Seek time slides tape to desired location; may be minutes
 - Read and/or write using head electromagnetic action
- Unix `tar` command is for fetch/store from a *tape archive*

Historic magnetic tape device



Video © George Adams, 07 July 2017. Filmed at the Computer History Museum, Mountain View, CA with the gracious help of Ken Ross, docent.

© 2018 by George B. Adams III

Tape specs yesterday and today

- 1952: first tape (IBM)
 - 1,400 bits/in² density; 2.3 Megabytes capacity on 10-inch reel
- Linear Tape-Open, a.k.a., LTO Ultrium
 - 2021 release of LTO-9: ninth generation, 5 more planned
 - Density increase > 140 billion times
 - Latency:
 - Load cartridge in a drive: ~ 15 sec
 - Unload cartridge from drive: ~ 20 sec
 - Average rewind: 60 sec
 - Average seek time: ~ 56 sec
 - Max Throughput: 400 MB/sec
 - Time to write a full tape at max speed: 12 hours 30 minutes
- Cost per bit is the lowest of all storage technologies today
 - LTO-9 cartridge (18 TB native / 45 TB compressed) \$94 @Walmart
- Tape relevant today for massive data archives



Tape advantages & disadvantages

■ Pro

- Least expensive storage technology
- Very dense

■ Con

- Very high latency
- Moderate power required for tape drive and possible robotic tape cartridge access mechanism

Possible future of memory technology

■ Future Flash

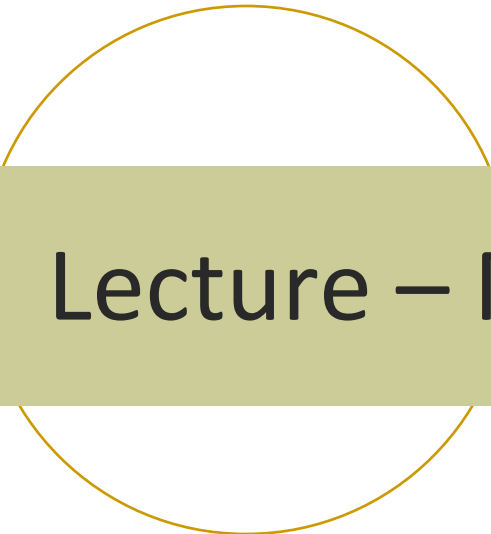
- Likely continuing price reduction will further squeeze hard disk drives out of performance-sensitive applications
- You may have already bought your last hard disk

■ RRAM – “Research” RAM

- Bit stored as changed resistance of molecule-sized region
- Likely non-volatile
- Latency perhaps just a few nanoseconds
- Might not be wear-limited
- Names: memristor, phase-change memory, conductive bridging RAM, ...
- Many ideas, nothing commercially available today

Summary

- Memory is now cheaper, denser, and lower power than ever before
- *However, only modest latency improvement*
- Moore's Law improvement of MOSFET logic gate circuit propagation delay for CPUs/GPUs/TPUs/xPUs far outpaced latency reduction of DRAM memory (transistor + capacitor) over the same period



Lecture – Memory addressing

A word to the wise is enough.
– Plautus, *Roman playwright*

Today's physical memory technologies

- Read chapter 5
- Learning objectives
 - Demonstrate conversion of a byte address to a word address
 - Explain how memory address pointers are decomposed into a word address field and a byte offset field
 - Explain the operation of memory

Processor time for memory access

- Present read or write request and pointer to memory controller
 - If read (fetch), memory latency visible to instruction
- Writes can be **buffered (queued)** to **hide latency**
 - Processor places write request in queue of registers, then moves to next machine instruction
 - Meanwhile, queue sends buffered writes to memory controller when latency permits
 - Store (write) instruction does not see memory latency unless queue is full

Memory transfer size (word or block size)

- Transfer size is number of bits transferred to/from memory simultaneously
 - Size is an important designer choice
 - Larger transfer size increases performance (bits/second)
- CPU **word size** is size of its general purpose registers
- Memory may be byte-addressed but number of bits transferred is always CPU word size or a power of two multiple of word size to increase bits/second transfer rate

Physical memory address & organization

- Each **physical memory address** corresponds to a CPU word
- Physical memory is organized as an array of words (higher performance than byte array)
- Hardware reads/writes at least an entire word at a time, to improve performance

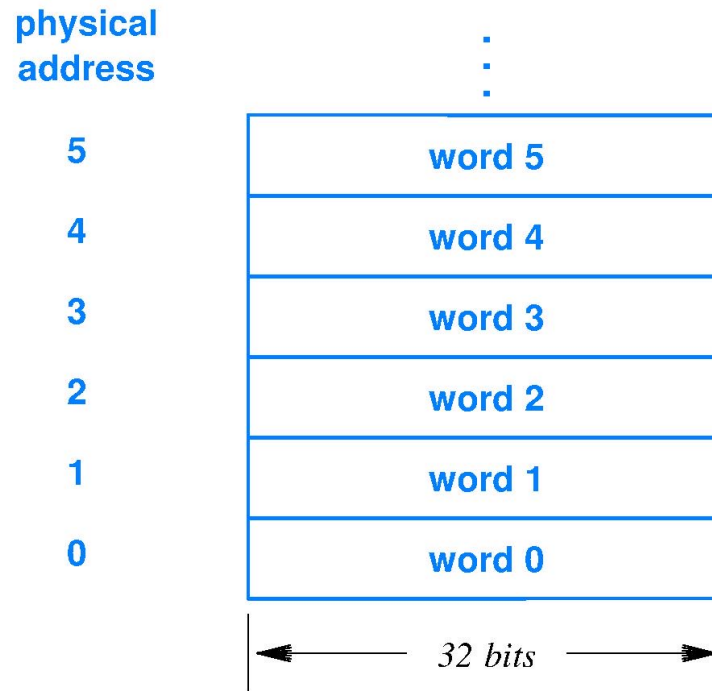


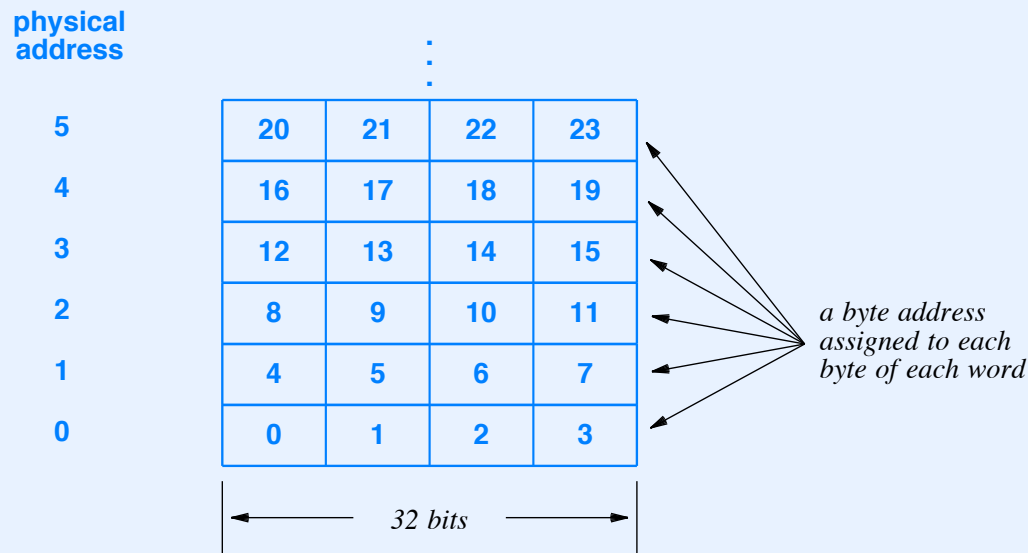
Figure 11.6 Physical memory addressing on a computer where a word is thirty-two bits. We think of the memory as an array of words.

Byte addressing and translation

- Byte addressing
 - View of memory presented to software
 - Each byte of memory is assigned an address
 - Convenient, traditional for programmers
- Memory controller
 - Translates byte addresses to word addresses

Example Of Address Translation

- Assume physical memory is organized into 32-bit words
- Programmer views memory as an array of bytes
- We think of each byte has having an address 0 through $N-1$
- Each physical word corresponds to 4 byte addresses



Given A Byte Address, B, Find The Byte

- Let N be the number of bytes per word
- The physical address of the word containing the byte is

$$W = \left\lfloor \frac{B}{N} \right\rfloor$$

- And byte the offset within the word is

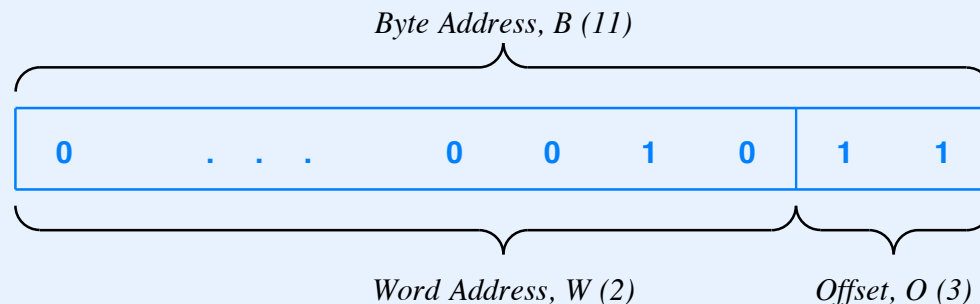
$$O = B \bmod N$$

- Example
 - Find byte $B = 11$ when $N = 4$
 - B can be found in word 2 at offset 3

$$W = \lfloor B/N \rfloor = \lfloor 11/4 \rfloor = 2 \quad O = B \bmod N = 11 \bmod 4 = 3$$

Fast ~~Efficient~~ Translation

- Think binary and choose word size N to be a power of 2
- Avoids arithmetic calculations, especially division and remainder
- Word address computed by extracting high-order bits
- Offset computed by extracting low-order bits
- Example: byte 11 with N equal to 4 bytes per word



$N = 2^2$ means Offset is 2 least significant bits,
Word is remaining more significant bits

Translation example with 0x notation

- Reminder: To convert bit string notation (0b) to hexadecimal notation (0x), start from rightmost bit and moving left, form 4-bit groups; pad final 4-bit group with 0 bits as needed; convert each group to its corresponding hex digit
- Example: map byte address 0x1234 to word address
 - Given word_size = 4 bytes, then offset_size = 2 bits and word address is 16 bits – 2 bits = 14 bits, must pad
 - Byte address 0x1234 = 0b0001 0010 0011 01**00**
 - Offset=0b**00**, word addr. = 0b0001 0010 0011 01
 - Padded word address = 0b0000001 0010 0011 01
 - Re-group padded addr. = 0b0000 0100 1000 1101
 - **Word address in hex = 0x 0 4 8 D**
 - *Byte addr. in hex again=0x 1 2 3 4*
 - **Conclusion: a little bit shifting radically alters hex notation**

Aligned objects in memory

- Designing for flexibility or performance when fetching multi-byte and 2^k -byte objects
 - Flexibility – Allow first byte of object to be at any byte address; can pack memory without unused bytes even when objects are not multiple-of-a-word sized
 - If object size not a multiple of word size, then there will be unused bytes (padding bytes)
 - Performance – First object byte must be aligned with the first byte of a physical word of memory

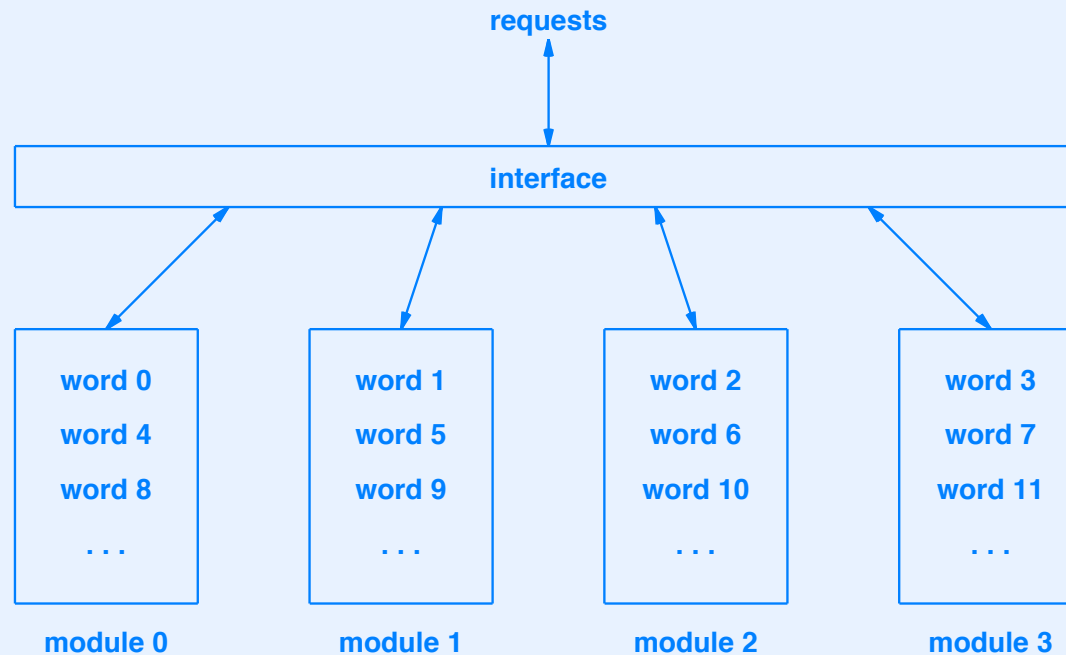
Increasing physical memory performance

- Make memory hardware more parallel, perform more accesses simultaneously
 - Memory banks
 - Memory interleaving – easy to improve performance if you code to produce favored patterns of access

Interleaving

- Related to memory banks
- Transparent to programmer
- Hardware places consecutive words (or consecutive bytes) in separate physical memories
- Technique: use low-order bits of address to choose module
- Known as *N-way interleaving*, where N is number of physical memories

Illustration Of 4-Way Interleaving



- Consecutive words stored in separate physical memories

Summary

- Physical memory
 - Organized into fixed-size words
 - Accessed through a controller
- Controller translates b'tw'n byte & word addressing
 - Byte addressing when communicating with a processor
 - Word addressing when communicating with a physical memory
- Avoid translation arithmetic, use powers of two for
 - Address space size
 - Bytes per word

Why have a memory hierarchy?

- Read chapter 5
- Learning objectives
 - Explain Amdahl's Law
 - Explain why the Rule of 90/10 for programs yields performance gain with caching is available

Why a hierarchy? Hardware reason

- No one memory technology satisfies these three memory criteria:

Fast Large Cheap

- However, a well-designed large and cheap **memory hierarchy** can have low latency much of the time

Why a hierarchy? Software reason

- **Program locality** – typically, for many, many consecutive fetch-execute cycles a program will have accessed only a fraction of its instructions and data
- Program locality has two dimensions
 - **Space**, a set of accessed addresses
 - **Time** – “for the next N clock cycles”
- If the space locality associated with a time locality fits in a small, fast memory, then fetch and store can be fast and, thus, fetch-execute can also be fast

90/10 program locality rule

- 90% of instructions executed – from start to finish of an application – come from just 10% of the listing instructions of the machine program
 - 90/10 program locality rule is true for many programs
- Data access is usually more evenly distributed over the individual items but often has considerable spacio-temporal locality

Effect of SRAM when 90/10 rule is true

- Consider program, P, having a listing containing M machine instructions, executed on computer, C
- Assume P obeys the 90/10 locality rule, then
 - SRAM memory with capacity 0.1M is added to C
 - This SRAM is 50 times faster than the DRAM in C
 - Assume the 0.1M machine instructions that are the 90% will migrate from the DRAM to the SRAM
- How much faster will C fetch instructions of P given the addition of SRAM and migration?

Amdahl's Law – speedup achieved OVERALL

Amdahl's Law models **overall speedup** accruing from an enhancement, which depends on two factors

- f_e = the fraction of the original time to complete task during which the enhancement can be used

$$f_e = \text{Fraction}_{\text{enhanced}} \leq 1$$

- S_e = the task speedup provided by the enhancement, i.e., ratio of speed using the enhancement to original unenhanced speed

$$S_e = \text{Speedup}_{\text{enhanced}} > 1$$

Amdahl's Law formula

$$\blacksquare \text{Speedup}_{\text{overall}} = \frac{\text{Original time of task}}{\text{Time using enhancement}}$$

$$\blacksquare S_{\text{overall}} = \frac{\text{Normalized original time}}{\text{Normalized time using enhancement}}$$

$$\blacksquare S_{\text{overall}} = \frac{1}{(1-f_e) + \frac{f_e}{S_e}}; \quad \begin{array}{l} 1 = \text{normalized original time} \\ f_e = \text{fraction enhanced} \\ S_e = \text{speedup of enhancement} \end{array}$$

$$\blacksquare S_{\text{overall}} = \frac{(1-f_e)}{(1-f_e)} + \frac{f_e}{\frac{f_e}{S_e}} = \frac{\text{Execution time}}{(1-f_e) + f_e} \div \frac{(1-f_e) + \frac{f_e}{S_e}}{(1-f_e) + f_e}$$

Normalized time of unenhanced portion of task before enhancement (numerator) and after (denominator)

Normalized time of enhanced portion of task before enhancement (numerator) and after (denominator)

Solve example with SRAM enhancement

- SRAM added to C, so SRAM is the enhancement
- Original time is with DRAM only
- SRAM 50 times faster than DRAM means $S_e = 50$
- P obeys 90/10 and the 10% stored in the SRAM make up 90% of instruction fetches means $f_e = 0.9$
- After SRAM is added how much faster does C fetch the instructions of P?
 - Using Amdahl's Law
 - $$S_{overall} = \frac{1}{(1-F_e) + \left(\frac{F_e}{S_e}\right)} = \frac{1}{1-0.9 + \frac{0.9}{50}} = \frac{1}{0.1 + 0.018} = \frac{1}{0.118}$$
 - $S_{overall} = 8.47$ times faster than DRAM alone

What is the hierarchy physical design?

- Read chapter 5
- Learning objective
 - Explain the memory hierarchy pyramids of technology, word and block movement, structure and addressing, and performance

Memory hierarchy concept

- Build levels of memory spanning a wide range of latency and capacity
- Goals
 - Average cost/bit close to the cost/bit of cheapest/bit technology
 - Average latency closer to the latency of the fastest technology, when 90/10 Rule program behavior applies

Cache memory organizational model

- Cache is high-speed, temporary store for a copy of some information from a larger, slower memory
- If cache has it, requested access completes fast

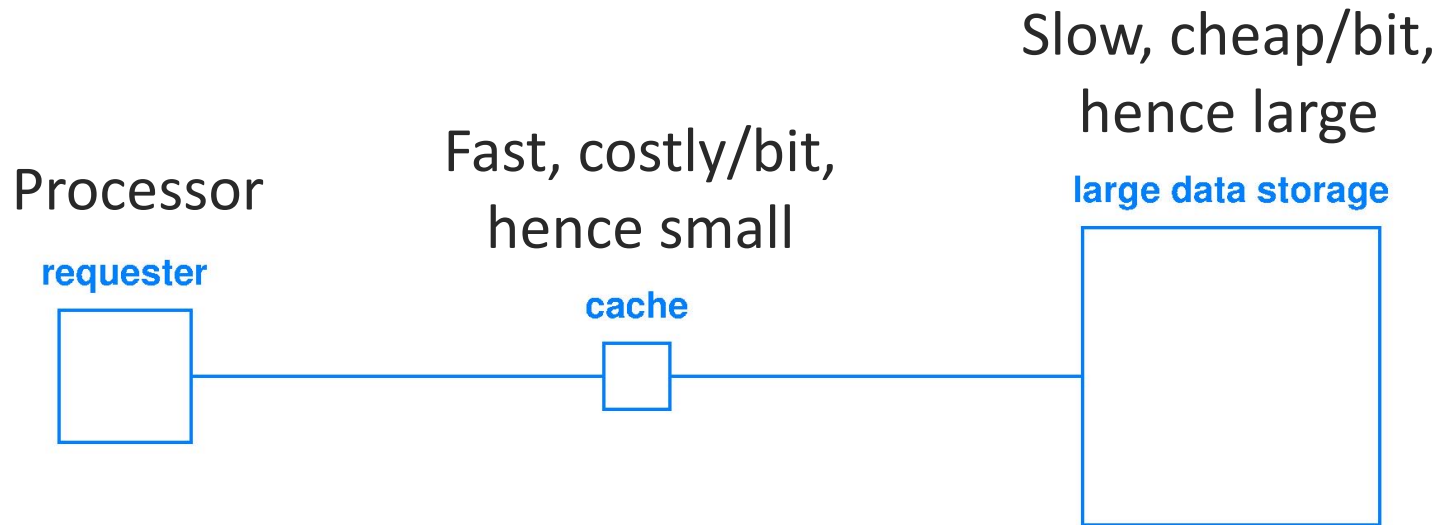
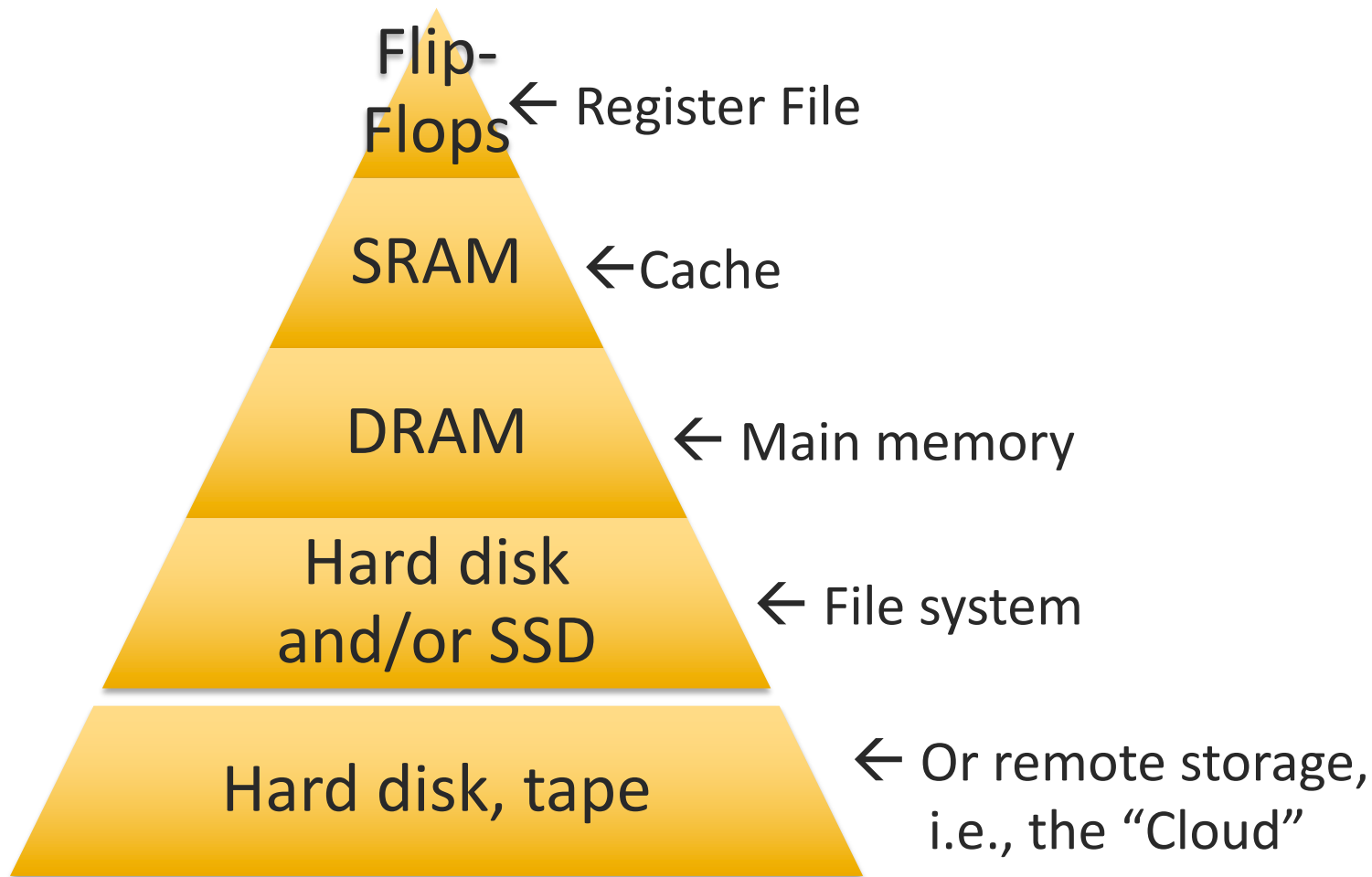


Figure 12.1 Conceptual organization of a cache, which is positioned on the path between a mechanism that makes requests and a storage mechanism that answers requests.

Example: achievement of hierarchy goals

- Goal: Average cost per bit close to cheapest
 - Ratio of SRAM cache capacity to DRAM main memory capacity typically 1:1000
 - Say SRAM costs 6/bit and DRAM costs 1/bit (based on memory cell transistor count, a proxy for chip area)
 - What is the average cost per bit in this SRAM and DRAM memory hierarchy?
 - Avg cost/bit = $(6*1+1*1000)/1001 = 1.005/\text{bit}$ ✓✓
- Goal: Average latency closer to lowest latency
 - From analysis using Amdahl's Law on previous slide
 $S_{overall} = 8.47$ times faster than DRAM and
only $50/8.47 = 5.9$ times slower than SRAM alone ✓

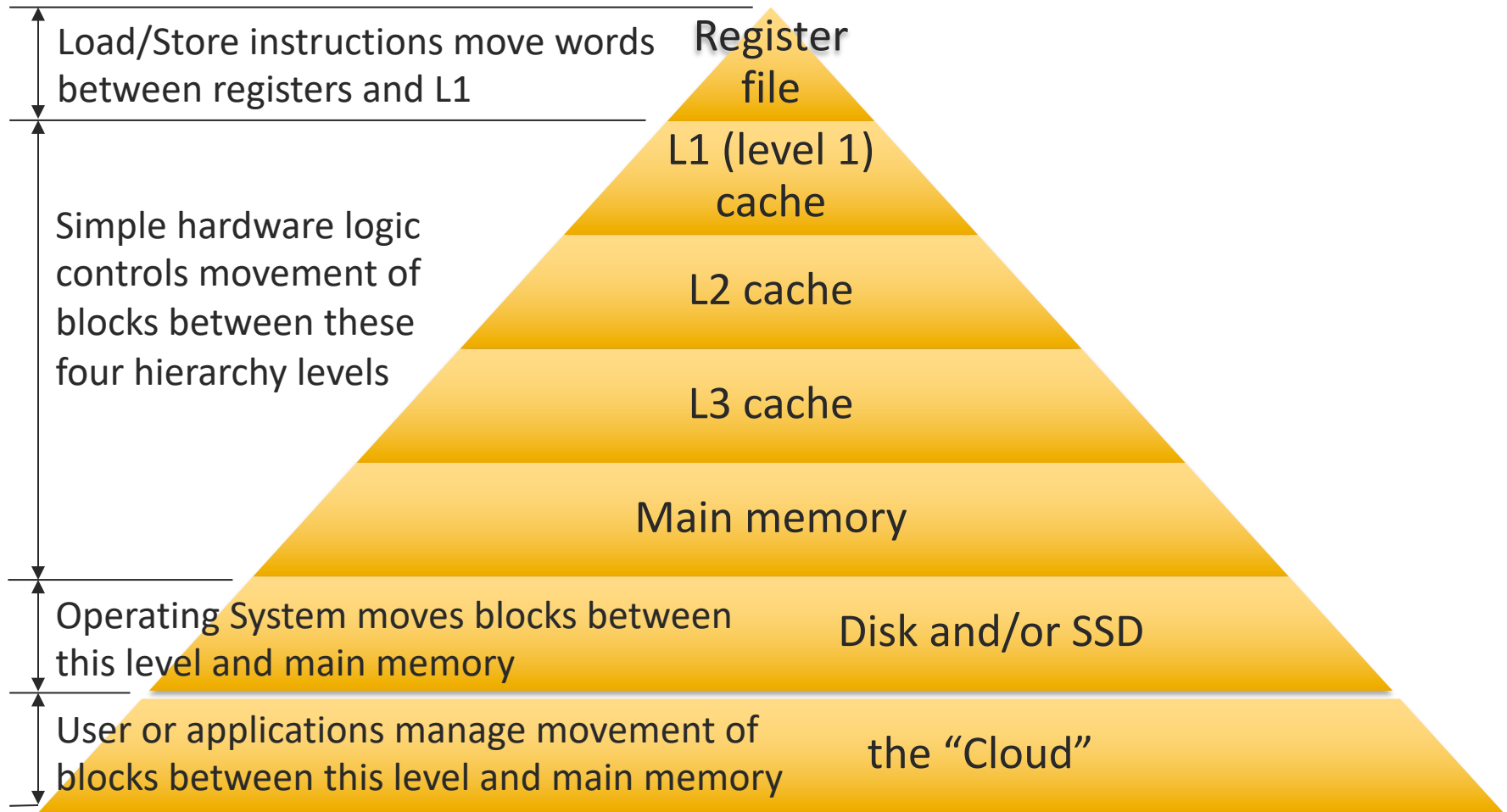
Memory hierarchy – levels by technology



Levels change as memory technologies come and go.

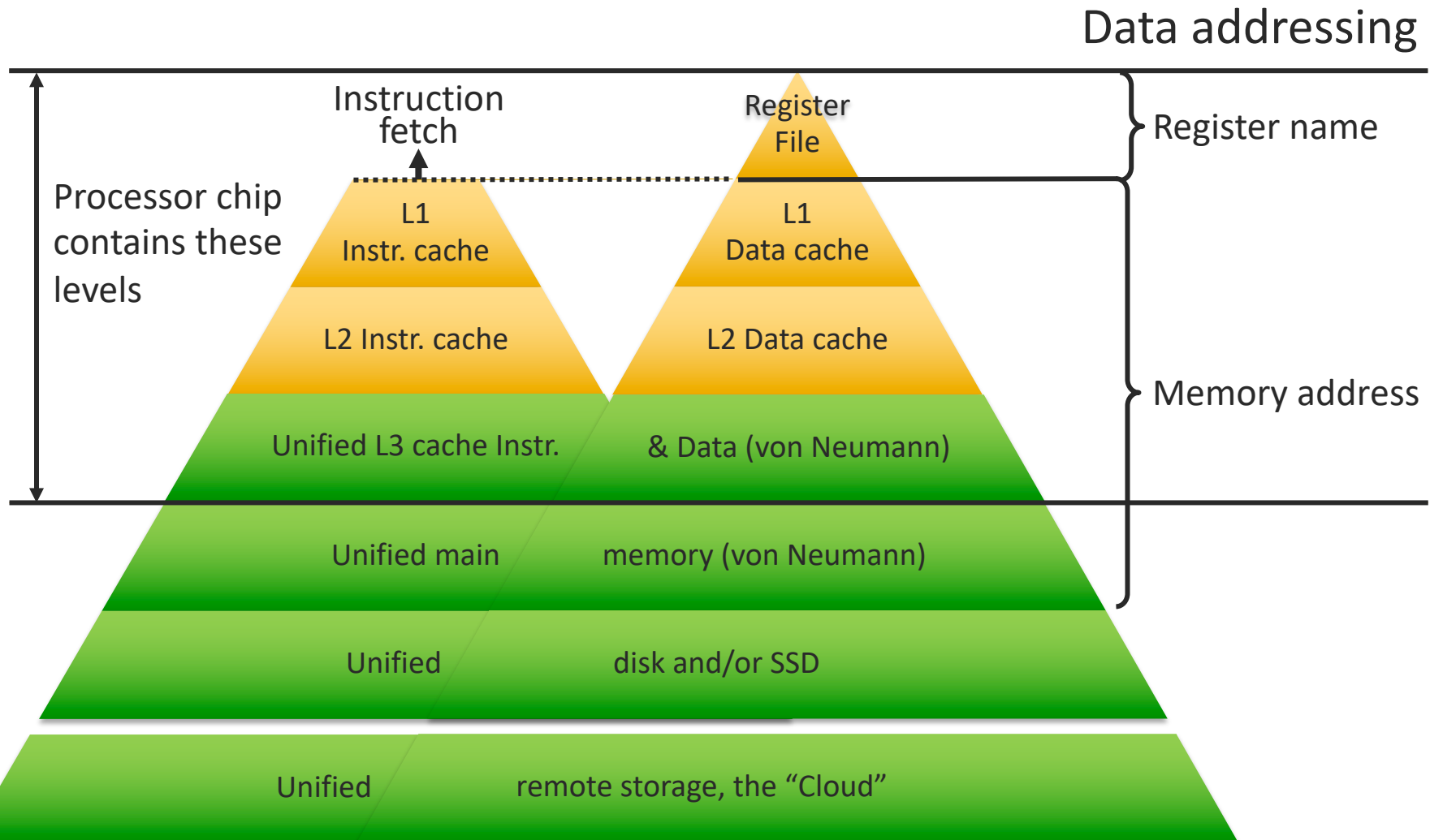
Width of level suggests capacity at that level but is NOT to scale.

Memory hierarchy – word and block movement

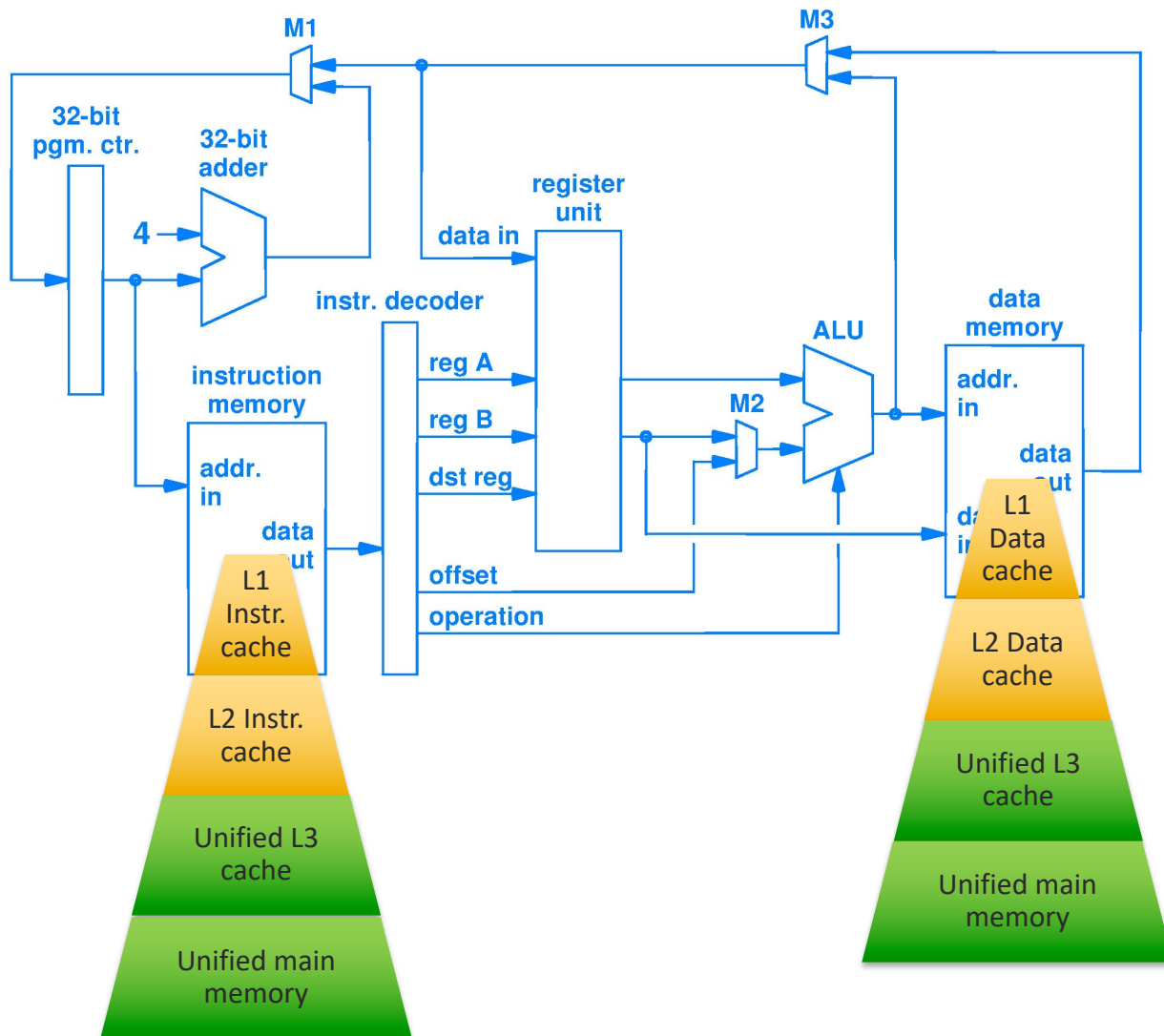


Memory hierarchy – structure & addressing

Both Harvard and von Neumann architectures

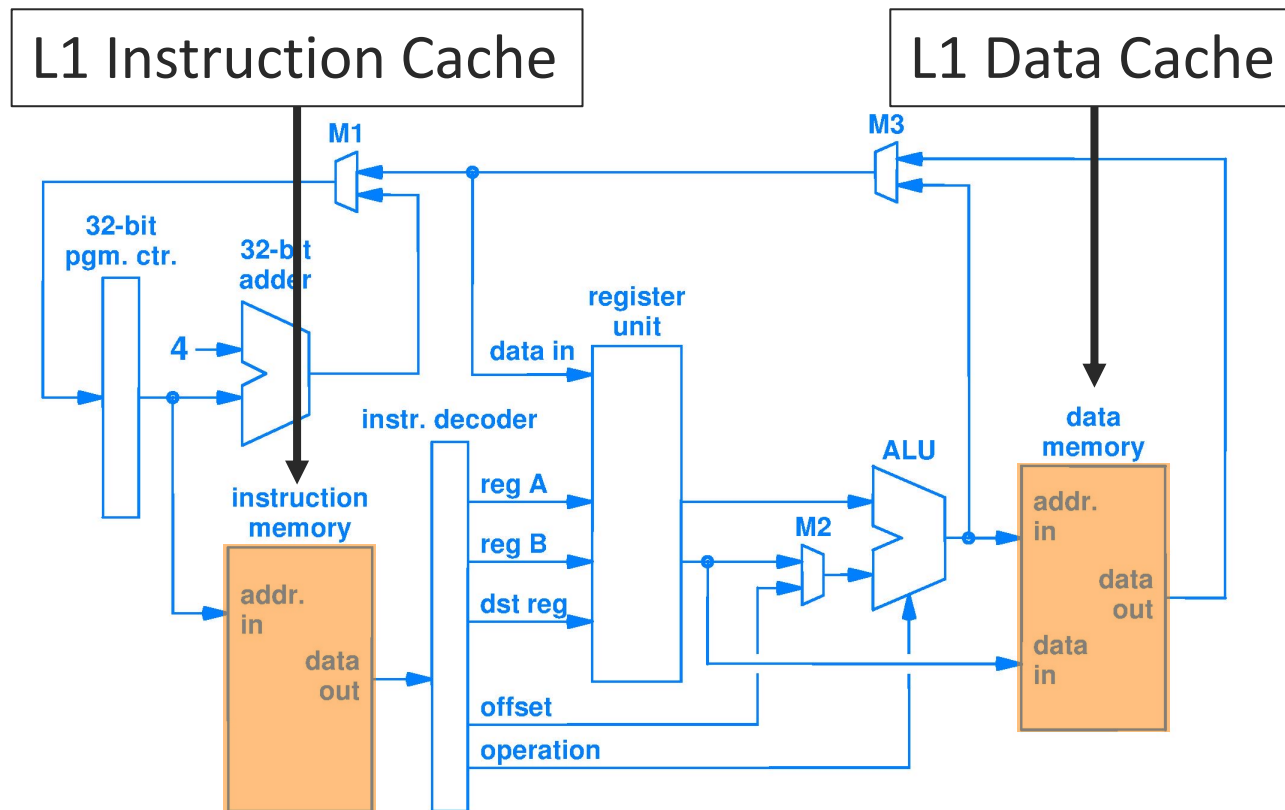


Processor and memory hierarchy



Memory hierarchy top levels appear in CPU

- Classic CPU circuit shows just the top level of the Harvard architecture memory hierarchy



Cache performance model

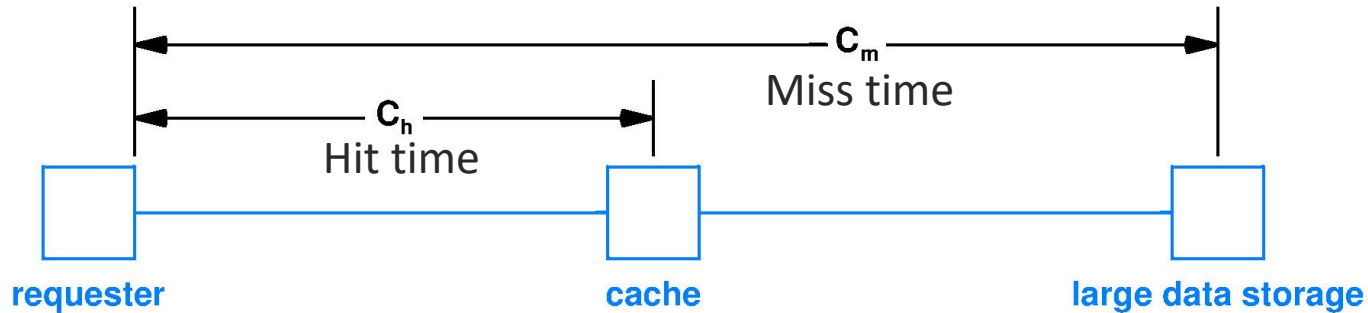
- Read chapter 5
- Learning objective
 - Explain AMAT (average memory access time) using the cache performance model

Cache terminology and policy

- Cache searched for every access request
 - Hit – successful search; cache processes access
 - Miss – unsuccessful search; request must be handled and be able to be handled by a lower level of the hierarchy
- Cache policy – store requested items and their address space neighbors to the extent possible
 - Hits depend on request sequence, called the **address trace**
 - First access by a new program is a guaranteed miss
 - Cache policy will fill the cache, what happens when full?

Cache performance model

■ Figure 12.2



■ Equation 12.5

Average memory access time (AMAT)
= Hit rate x Hit time + (1-Hit rate) x Miss time

Improving cache performance

- $AMAT = \text{Hit rate} \times \text{Hit time} + (1 - \text{Hit rate}) \times \text{Miss time}$
- Cache optimizations can usefully be conceived and organized according to which term and/or factor of the above equation is improved
 - Increase hit rate (Amdahl's Law focus, increase f_e)
 - Reduce hit time (improve worst case propagation delay of cache circuit design, i.e., faster hardware)
 - Reduce miss time (improve memory system design and/or improve memory technology)
- Using models to describe a system helps reveal all the ways to improve that system